UNIVERSITI SAINS MALAYSIA

SCHOOL OF COMPUTER SCIENCES

SEMESTER 2, 2025/2026

CPT411 - AUTOMATA THEORY & FORMAL LANGUAGES

ASSIGNMENT 1

LECTURER'S NAME:

ASSOCIATE PROFESSOR DR. GAN KENG HOON

NAME	MATRIC NO.
MIZAN QISTINA BINTI NORUDEN	160355
NORITA BINTI MUIN	160453
NUR SYAFIQAH BINTI MOHD YUSOF	160444

DATE OF SUBMISSION:

27th APRIL 2026

Table of Contents

1. Introduction

The objective of this assignment is to design and implement a Deterministic Finite Automaton (DFA) for certain case. In our case, we were assigned to produce program that capable of recognizing high-frequency English stop words within a text stream. In text processing, ingesting high volume of text input usually requires high computation. To address this, texts consist of information with fillers better to pre-processed to extract out the stop words or "function words" which does not carry high contextual meaning. This steps frequently done in natural language processing (NLP) to reduce text dimensionality simultaneously reduce the computation power.

The DFA for this project was formally designed and validated using JFLAP 7.1, a graphical tool for designing and simulating finite automata. The resulting automaton was then implemented as a web-based program using HTML, CSS, and vanilla JavaScript, accessible locally via any modern browser without requiring a server or additional dependencies.

1.1. Language Definition

The language assigned for this project is L6 which is about English Stop Words. The formal language defined as:

$L = \{w \in \Sigma^* \mid w$ is a complete token belongs to the predefined set of English stop words$\}$,

where alphabet, Σ = {a–z, A–Z, 0–9, and all other symbols found in the sample text}. The language, L is defined as the set of strings matching the 30 identified stop words.

The recogniser programmed to scan a stream of input text and identifies every token that is a stop word, reporting its position, frequency, and the DFA state trace used to reach the decision whether it is accepted or rejected.

1.2. Scope

The scope is limited to recognising 30 predefined English stop words using a single unified DFA. Stop words are extremely common words that carry little standalone semantic meaning such as articles, conjunctions, prepositions, pronoun and auxiliary verb. The complete stop word set is:

- **Article:** a, an, the,
- **Conjunction:** and, but, if, or, so, as
- **Preposition:** at, by, in, of, on, to,
- **Pronoun:** he, I, it, its, me, my, she, them, they, us, we
- **Auxiliary Verb:** is, are, was, were

The recognition is case-insensitive. To ensure the DFA remains comprehensive yet concise, all input characters are normalized to lowercase before state transition, allowing a single set of states to recognize 'The', 'THE', and 'the'. The recognizer matches whole tokens only. Partial matches within longer words are rejected because token boundaries trigger an acceptance check that requires the normalized token to be an exact member of the stop word set.

The system utilizes a tokenized DFA approach, where input text is pre-processed into word tokens, and each token is then validated character-by-character against a formal state transition designed in JFLAP. By doing the tokenization, this help the complexity on the transition to only validate the character involved rather than the character with the delimiters like whitespace, coma, and period with loop to initial state to scan new word.

1.3. Complete DFA

The automaton has the following formal definition:

M = (Q, Σ, δ, q0, F) where:

- **Q** = {q0, q1, q2, …, q37} — 38 states total
- **Σ** = all printable ASCII characters (a–z, A–Z, 0–9, and all other symbols)
- **δ**: **Q × Σ → Q** — (***d (q, x) = q1)***
- **q0** = initial state (single start state)
- **F** = {q1, q3, q4, q5, q6, q7, q10, q11, q12, q14, q15, q19, q21, q23, q25, q27, q28, q29, q30, q31} — 20 final or accept states

Transition function defines in the Table 2 Complete State Transition Function listing all transitions in the form δ(q, x) = q. The table can be found in 4.1 Appendix A: DFA Design.

Trap states q32–q37 each loop on all input characters and are never accept states. Once the DFA enters a trap state, processing of the current token terminates immediately as per the assignment requirement. The complete DFA diagram of the stop words recognizer created using JFLAP 7.1 is provided in 4.1 Appendix A (Figure 1).

2. Implementation Information

2.1. String Reading and Processing

The program strictly processes one character at a time from left to right, as required by the assignment specification. No built-in string-searching or pattern-matching functions are used at any point in the pipeline. Instead, the implementation relies entirely on indexed character access, ASCII arithmetic, and the DFA transition table to drive all processing decisions. The pipeline consists of five stages as described below.

a) Stage 1: Character-by-character tokenisation

The input text uses an integer index, *i*. At each position, *text.charCodeAt(i)* returns the ASCII code of exactly one character. A custom *isAlphaCode()* function checks whether the code is in the range 65-90 (uppercase) or 97-122 (lowercase). When an alphabetic character is detected, subsequent characters are appended one by one into a token buffer until a non-alphabetic delimiter like whitespace, comma, or period is encountered.

To maintain the rigor of a true DFA simulation, functions like *String.split(), indexOf(), includes(), match(), replace(),* and all regular expressions are completely absent from the code.

b) Stage 2: Per-character case normalisation

Furthermore, case-insensitivity is handled at the character level rather than the string level to ensure the DFA remains mathematically precise. The custom *lowerCharCode()* function checks if a character code falls within the uppercase range (65–90) and adds 32 to obtain the lowercase equivalent.

This manual transformation ensures that the built-in *String.toLowerCase()* is never applied to tokens or the full text, fulfilling the requirement for low-level character manipulation.

c) Stage 3: DFA simulation

The function *runDFAOnToken()* begins at state q0 and reads each character of the token one at a time. For every character, *transition()* looks up *δ(currentState, ch)* in the DELTA table or transition table. If an explicit entry for the character exists, that next state is taken. Otherwise the 'others' entry for the current state is followed, which leads to a trap state. If a trap state is reached, the loop terminates immediately.

d) Stage 4: Output

For every token the program records the original word, the character position in the text, the token index, accept/reject status, the full DFA state trace *(e.g. q0 --i--> q28 --*

t--> q30), and the final state. Four output tabs are produced including, annotated text with stop words highlighted in bold colour, a complete DFA trace log, position of words and an occurrence frequency. Figure 2-7 shows the interface and the result of the stop words recognition.

2.2. Programming Constructs Used

This part describes the programming construct used to implement the DFA-based Stop Words Finder. including the (simplified) code snippet. Each construct is explained in terms of its role and purpose within the system. The code snippets for each construct are provided for reference in 4.2 Appendix B.

a) The State Transition Table (DELTA)

The program starts by defining the *DELTA* object which act as the hardcoded DFA transition table. Each of the key will represent a state such as q0. The value is an object where the keys represent a specific character input, and the values represent the next state for current state. This part of the code serves as the mapping path that the program must follow when it reads the inputs.

b) Acceptance Criteria (ACCEPT_STATES & STOP_SET)

The acceptance criteria are defined through the *ACCEPT_STATES* and *STOP_SET* objects. The *ACCEPT_STATES* object acts as a finish line marker where it identifies which states are considered valid endpoints for a word. If the words final letter ends up on one of the *ACCEPT_STATE,* the words will be considered being accept as a stop word. The *STOP_SET* object is also included in the program to prevent any false positive. It performs a final string-to-key comparison after the DFA has finished its run.

c) Manual Character Helpers

To follow the low-level processing principle, the program used manual helper functions for the input characters such as *lowerCharCode* and *isAlphaCode*. Instead of using a built-in code like *toLowerCase(),* we use the *lowerCharCode* to manually manipulates character cases based on their ASCII values by adding 32 to it. Furthermore, the *isAlphaCode* is used to check whether a particular character is an alphabetical letter. This function is used to detect the word boundary inside the text. These concepts are important to check everything character-by-character to ensure the program follow the basic processing principle.

d) Lexical Tokenization Logic (tokenizer function)

The tokenizer function acts as the pre-processing engine where it prepares the raw data into tokens for the DFA inputs. The function will scan the sequentially using loop

to transverse the entire input string character by character. Inside the loop, the program used nested condition logic to determine whether the character is part of the words or not. For example, symbol like punctuation and space does not consider as part of a word. When a letter is detected, the program "looks ahead" to group all consecutive letters into a single "word" token. This part of the code is very important to isolate the strings that will be evaluated by the state machine. It is to ensure that the DFA only process meaningful sets of string while maintaining the layout of the input texts.

e) State Transition Logic (transition function)

The transition function acts as the decision-making logic that connects the input character to the DELTA object. This function will take the current state and read the specific character to determine the next destination of the transition. It first uses conditional logic to check for an explicit match in the transition table. If there is no match found, it will fall back to an “others” key or trap state. This function is important to ensure the program remain deterministic. The program will take very single character input and use it to find exactly one predictable state to move next.

f) The DFA Execution Engine (runDFAOnToken)

The runDFAOnToken function acts as the execution engine of the system. The execution engine will implement mathematical logic of the automaton. For each of the word that were identified by tokenizer, the system will initially set the current state as q0 which is the start state. The process then will enter a deterministic loop that iterates through the word's characters. During each iteration, the program will call for transition function to move the current state to the next state based on the input character. This process includes a tracing construct that records every transition state into an array. This trace of the transtion allows user to see the exact logical path that the program has traveled before it reach the final state whether the word being “Accept” or “Reject” by the program.

g) HTML Escaping and UI Rendering

The final part of the program handles the visual output through the escapeHTML function and the rendering logic. Because the input text might contain special characters such as < or & that could interfere with the browser, the escapeHTML function manually checks each character and converts it into a safe format. After cleaning the text, the program uses the results from the DFA to rebuild the string, wrapping any identified stop words in <mark> tags. This allows the program to highlight the results in the browser while ensuring the original text is displayed safely and accurately without using risky built-in shortcuts.

Table 1 below summarizes all the programming constructs discussed above, providing a concise overview of each construct, its implementation in code, and its purpose within the system.

Table 1 Summary of Programming Constructs Used

Program Construct	Implementation in Code	Purpose
State Transition Table	DELTA object	Acts as the hardcoded map for all possible state movements.
Acceptance Criteria	ACCEPT_STATES and STOP_SET object	Defines the "finish line" and provides final word verification.
Manual Character Helpers	LowerCharCode and isAlphaCode	Handles low-level ASCII math to follow strict char-by-char rules.
State Transition Logic	transition(state, ch)	The decision-making gate that looks up the next state for every character.
Lexical Tokenizer	tokenize(text)	Pre-processes raw text to isolate meaningful words for the machine.
DFA Execution Engine	runDFAOnToken(chars[])	Drives the deterministic loop and records the logical trace path.
HTML Escaping and UI Rendering	escapeHTML(str)	Cleans the output and visually highlights identified stop words.

2.3. User Interface and Results

The output is delivered through a web-based interface built using HTML, CSS, and vanilla JavaScript, accessible locally via any browser without requiring a server or additional dependencies. The interface consists of three tabbed panels. This includes, annotated output, DFA trace log, positions and occurrence frequency table, providing a clear and interactive demonstration of the recognizer's behavior. Figure 2 shows the main interface run locally.

User can simply paste their sample text in the input text box, then click the run text button to view the results. Figure 3-6 (see 4.3 Appendix C) shows the stop words recognizer's result of each tab by using sample text I.

3. Conclusion

This project successfully implements a Deterministic Finite Automaton-based recogniser for English stop words, satisfying all requirements of the CPT411 assignment. The DFA was formally designed in JFLAP 7.1, producing a 38-state automaton capable of recognising exactly 30 English stop words. The stop words span for five grammatical categories that are articles, prepositions, pronouns, conjunctions and verb.

The implementation rigorously processes input one character at a time. Every stage of the pipeline which are tokenisation, case normalisation, DFA simulation, and output generation avoids all string-searching APIs. Custom helper functions replace every prohibited operation, ensuring the program accurately simulates a finite state machine as required.

The web-based interface provides a clear and interactive demonstration. Four output panels are provided. This include, annotated text with stop words visually highlighted, a complete DFA state trace for every token using the exact q-labels from the JFLAP design, word position table, and a frequency table with occurrence counts and relative percentages. This satisfy the assignment's requirement for additional information output.

4. Appendix

4.1. Appendix A: DFA Design

a) Complete DFA Diagram

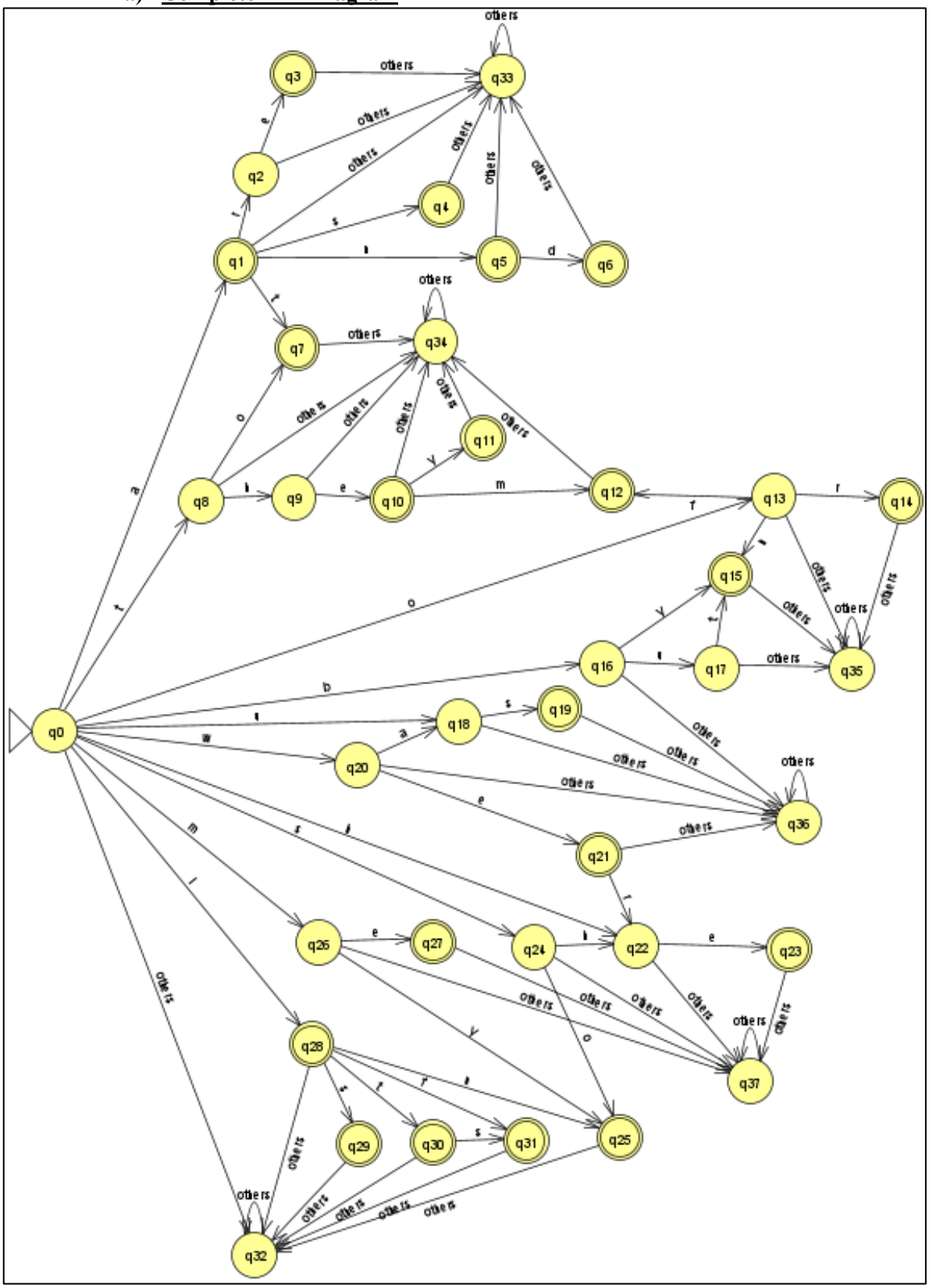

Figure 1 Full DFA Diagram

b) Complete State Transition Table

Table 2 Complete State Transition Table listing all transitions in the form δ(q, x) = q'

From State, *q*	Input Character, *x*	To State, *q'*	Transition Function, *δ (q, x) = q'*
q0	*a*	*q1*	*δ (q0, a) = q1*
q0	*t*	*q8*	*δ (q0, t) = q8*
q0	*o*	*q13*	*δ (q0, o) = q13*
q0	*b*	*q16*	*δ (q0, b) = q16*
q0	*u*	*q18*	*δ (q0, u) = q18*
q0	*w*	*q20*	*δ (q0, w) = q20*
q0	*h*	*q22*	*δ (q0, h) = q22*
q0	*s*	*q24*	*δ (q0, s) = q24*
q0	*m*	*q26*	*δ (q0, m) = q26*
q0	*i*	*q28*	*δ (q0, i) = q28*
q0	*others*	*q32*	*δ (q0, others) = q32*
q1	*r*	*q2*	*δ (q1, r) = q2*
q1	*s*	*q4*	*δ (q1, s) = q4*
q1	*n*	*q5*	*δ (q1, n) = q5*
q1	*t*	*q7*	*δ (q1, t) = q7*
q1	*others*	*q33*	*δ (q1, others) = q33*
q2	*e*	*q3*	*δ (q2, e) = q3*
q2	*others*	*q33*	*δ (q2, others) = q33*
q3	*others*	*q33*	*δ (q3, others) = q33*
q4	*others*	*q33*	*δ (q4, others) = q33*
q5	*d*	*q6*	*δ (q5, d) = q6*
q5	*others*	*q33*	*δ (q5, others) = q33*
q6	*others*	*q33*	*δ (q6, others) = q33*
q7	*others*	*q34*	*δ (q7, others) = q34*
q8	*o*	*q7*	*δ (q8, o) = q7*
q8	*h*	*q9*	*δ (q8, h) = q9*
q8	*others*	*q34*	*δ (q8, others) = q34*
q9	*e*	*q10*	*δ (q9, e) = q10*
q9	*others*	*q34*	*δ (q9, others) = q34*
q10	*y*	*q11*	*δ (q10, y) = q11*
q10	*m*	*q12*	*δ (q10, m) = q12*
q10	*others*	*q34*	*δ (q10, others) = q34*
q11	*others*	*q34*	*δ (q11, others) = q34*

q12	*others*	*q34*	*δ (q12, others) = q34*
q13	*r*	*q14*	*δ (q13, r) = q14*
q13	*f*	*q12*	*δ (q13, f) = q12*
q13	*n*	*q15*	*δ (q13, n) = q15*
q13	*others*	*q35*	*δ (q13, others) = q35*
q14	*others*	*q35*	*δ (q14, others) = q35*
q15	*others*	*q35*	*δ (q15, others) = q35*
q16	*y*	*q15*	*δ (q16, y) = q15*
q16	*u*	*q17*	*δ (q16, u) = q17*
q16	*others*	*q36*	*δ (q16, others) = q36*
q17	*t*	*q15*	*δ (q17, t) = q15*
q17	*others*	*q35*	*δ (q17, others) = q35*
q18	*s*	*q19*	*δ (q18, s) = q19*
q18	*others*	*q36*	*δ (q18, others) = q36*
q19	*others*	*q36*	*δ (q19, others) = q36*
q20	*a*	*q18*	*δ (q20, a) = q18*
q20	*e*	*q21*	*δ (q20, e) = q21*
q20	*others*	*q36*	*δ (q20, others) = q36*
q21	*r*	*q22*	*δ (q21, r) = q22*
q21	*others*	*q36*	*δ (q21, others) = q36*
q22	*others*	*q37*	*δ (q22, others) = q37*
q22	*e*	*q23*	*δ (q22, e) = q23*
q23	*others*	*q37*	*δ (q23, others) = q37*
q24	*h*	*q22*	*δ (q24, h) = q22*
q24	*o*	*q25*	*δ (q24, o) = q25*
q24	*others*	*q37*	*δ (q24, others) = q37*
q25	*others*	*q32*	*δ (q25, others) = q32*
q26	*e*	*q27*	*δ (q26, e) = q27*
q26	*y*	*q25*	*δ (q26, y) = q25*
q26	*others*	*q37*	*δ (q26, others) = q37*
q27	*others*	*q37*	*δ (q27, others) = q37*
q28	*s*	*q29*	*δ (q28, s) = q29*
q28	*t*	*q30*	*δ (q28, t) = q30*
q28	*f*	*q31*	*δ (q28, f) = q31*
q28	*n*	*q25*	*δ (q28, n) = q25*
q28	*others*	*q32*	*δ (q28, others) = q32*

q29	*others*	*q32*	*δ(q29, others) = q32*
q30	*s*	*q31*	*δ(q30, s) = q31*
q30	*others*	*q32*	*δ(q30, others) = q32*
q31	*others*	*q32*	*δ(q31, others) = q32*
q32	*others*	*q32*	*δ(q32, others) = q32*
q33	*others*	*q33*	*δ(q33, others) = q33*
q34	*others*	*q34*	*δ(q34, others) = q34*
q35	*others*	*q35*	*δ(q35, others) = q35*
q36	*others*	*q36*	*δ(q36, others) = q36*
q37	*others*	*q37*	*δ(q37, others) = q37*

4.2. Appendix B: Key Source Code Segments

A complete source code for the DFA Stop Words Finder web interface is available on GitHub. Below is the repository URL:

https://github.com/mnnrita03/CPT411-ASSG1-STOPWORDSRECOGNIZER.git

a) DELTA Transition Table

```
const DELTA = {
  // start state
  'q0':  { 'a':'q1',  'b':'q16', 'h':'q22', 'i':'q28', 'm':'q26',
           'o':'q13', 's':'q24', 't':'q8',  'u':'q18', 'w':'q20',
           'others':'q32' },

  // a-branch
  'q1':  { 'n':'q5',  't':'q7',  'r':'q2',  's':'q4',  'others':'q33' },
  'q2':  { 'e':'q3',  'others':'q33' },
  'q3':  { 'others':'q33' },
  'q4':  { 'others':'q33' },
  'q5':  { 'd':'q6',  'others':'q33' },
  'q6':  { 'others':'q33' },
  'q7':  { 'others':'q34' },

  //  i-branch
  'q28': { 'f':'q31', 'n':'q25', 't':'q30', 's':'q29', 'others':'q32' },
  'q30': { 's':'q31', 'others':'q32' },
  'q31': { 'others':'q32' },
  'q29': { 'others':'q32' },
  'q25': { 'others':'q32' },

  // t-branch
  'q8':  { 'o':'q7',  'h':'q9',  'others':'q34' },
  'q9':  { 'e':'q10', 'others':'q34' },
  'q10': { 'y':'q11', 'm':'q12', 'others':'q34' },
  'q11': { 'others':'q34' },
  'q12': { 'others':'q34' },

  // h-branch
  'q22': { 'e':'q23', 'others':'q37' },
  'q23': { 'others':'q37' },

  // s-branch
```

```
  'q24': { 'o':'q25', 'h':'q22', 'others':'q37' },

  // o-branch
  'q13': { 'r':'q14', 'f':'q12', 'n':'q15', 'others':'q35' },
  'q14': { 'others':'q35' },
  'q15': { 'others':'q35' },

  // b-branch
  'q16': { 'y':'q15', 'u':'q17', 'others':'q36' },
  'q17': { 't':'q15', 'others':'q35' },

  // u-branch
  'q18': { 's':'q19', 'others':'q36' },
  'q19': { 'others':'q36' },

  // w-branch
  'q20': { 'e':'q21', 'a':'q18', 'others':'q36' },
  'q21': { 'r':'q22', 'others':'q36' },

  // m-branch
  'q26': { 'e':'q27', 'y':'q25', 'others':'q37' },
  'q27': { 'others':'q37' },

  // trap states (loop on everything)
  'q32': { 'others':'q32' },
  'q33': { 'others':'q33' },
  'q34': { 'others':'q34' },
  'q35': { 'others':'q35' },
  'q36': { 'others':'q36' },
  'q37': { 'others':'q37' },
};
```

b) Acceptance Criteria

```
// Accept states
const ACCEPT_STATES = {
  'q1':true,'q3':true,'q4':true,'q5':true,'q6':true,'q7':true,
  'q10':true,'q11':true,'q12':true,'q14':true,'q15':true,
  'q19':true,'q21':true,'q23':true,'q25':true,'q27':true,
  'q28':true,'q29':true,'q30':true,'q31':true
};

// The valid stop words – used only for boundary acceptance check (1- true)
const STOP_SET = {
  'a':1,'an':1,'and':1,'at':1,
  'by':1,'but':1,
  'he':1,
  'i':1,'if':1,'in':1,'it':1,'its':1,
  'me':1,'my':1,
  'of':1,'on':1,'or':1,
  'she':1,'so':1,
  'the':1,'them':1,'they':1,'to':1,
  'us':1,
  'we':1,
  'is':1,'are':1,'was':1,'were':1,'as':1
};
```

c) Single-character case fold (Manual Character Helper)

```
function lowerCharCode(code) {
  // A=65..Z=90 → a=97..z=122
  if (code >= 65 && code <= 90) return code + 32;
  return code;
}

function isAlphaCode(code) {
  return (code >= 65 && code <= 90) || (code >= 97 && code <= 122);
}
```

d) Character-by-character tokeniser (Lexical Tokenizer)

```
function tokenize(text) {
  const tokens = [];
  let i = 0;
  while (i < text.length) {
    const code = text.charCodeAt(i);
    if (isAlphaCode(code)) {
      const chars   = [];
      const display = [];
      const start   = i;
      while (i < text.length && isAlphaCode(text.charCodeAt(i))) {
        chars.push(text.charAt(i));
        display.push(text.charAt(i));
        i++;
      }
      tokens.push({ chars: chars, display: display, start: start, type: 'word' });
    } else {
      tokens.push({ chars: [text.charAt(i)], display: [text.charAt(i)], start: i, type: 'other' });
      i++;
    }
  }
  return tokens;
}
```

e) State Transition Logic

```
function transition(state, ch) {
  const row = DELTA[state];
  if (!row) return 'q33';                // unknown state → trap
  // Check explicit char label first
  const lower = String.fromCharCode(lowerCharCode(ch.charCodeAt(0)));
  if (row[lower] !== undefined) return row[lower];
  // Fall through to 'others'
  if (row['others'] !== undefined) return row['others'];
  return 'q33';
}
```

f) DFA Execution Engine

```
function runDFAOnToken(charArr) {
  let state = 'q0';
  const steps = [];
  // Build normalised word char-by-char for boundary check
  const normArr = [];

  for (let i = 0; i < charArr.length; i++) {
    const ch   = charArr[i];
    const from = state;
    const to   = transition(state, ch);
    steps.push({ ch: ch, from: from, to: to });
    state = to;
    // accumulate lowercase char for boundary check
    normArr.push(String.fromCharCode(lowerCharCode(ch.charCodeAt(0))));
  }

  // Build normalised word string (only safe join of our own array)
```

```
  let normWord = '';
  for (let i = 0; i < normArr.length; i++) normWord += normArr[i];

  // Accept iff final state is in ACCEPT_STATES AND word is a known stop word
  const accepted = (ACCEPT_STATES[state] === true) && (STOP_SET[normWord] === 1);

  return { accepted: accepted, steps: steps, finalState: state, normWord: normWord };
}
```

g) HTML Escaping and UI Rendering

```
function escapeHTML(str) {
  let out = '';
  for (let i = 0; i < str.length; i++) {
    const ch = str.charAt(i);
    if      (ch === '&') out += '&amp;';
    else if (ch === '<') out += '&lt;';
    else if (ch === '>') out += '&gt;';
    else                 out += ch;
  }
  return out;
}
```

4.3. Appendix C: User Interface

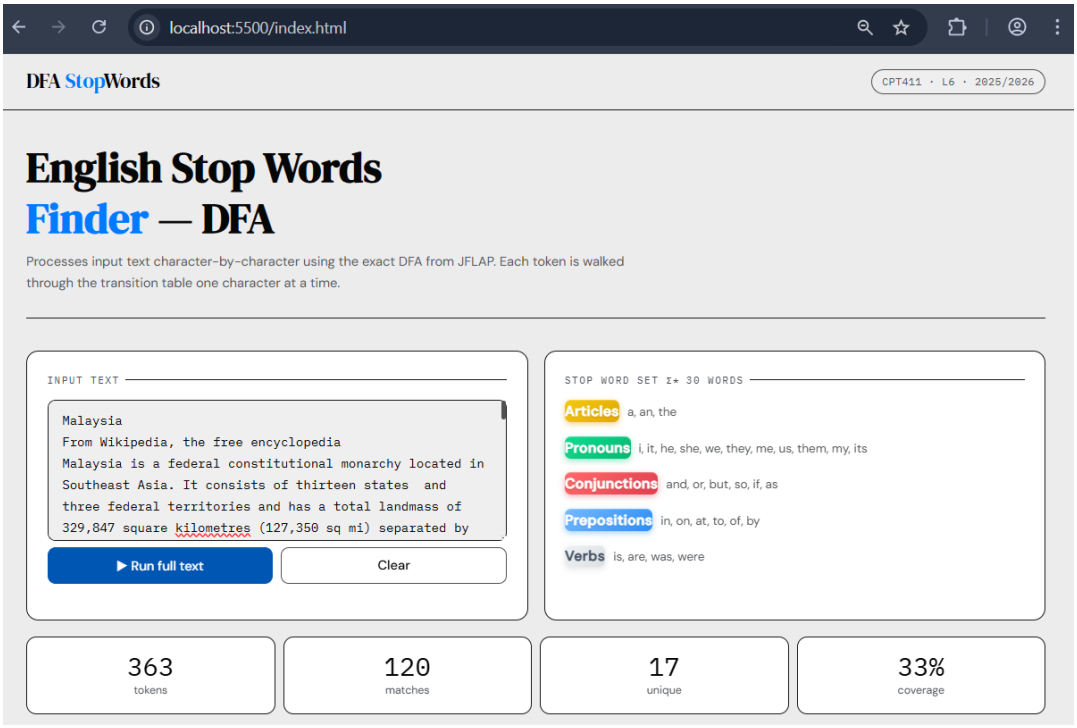

Figure 2 Interface with Sample Text 1

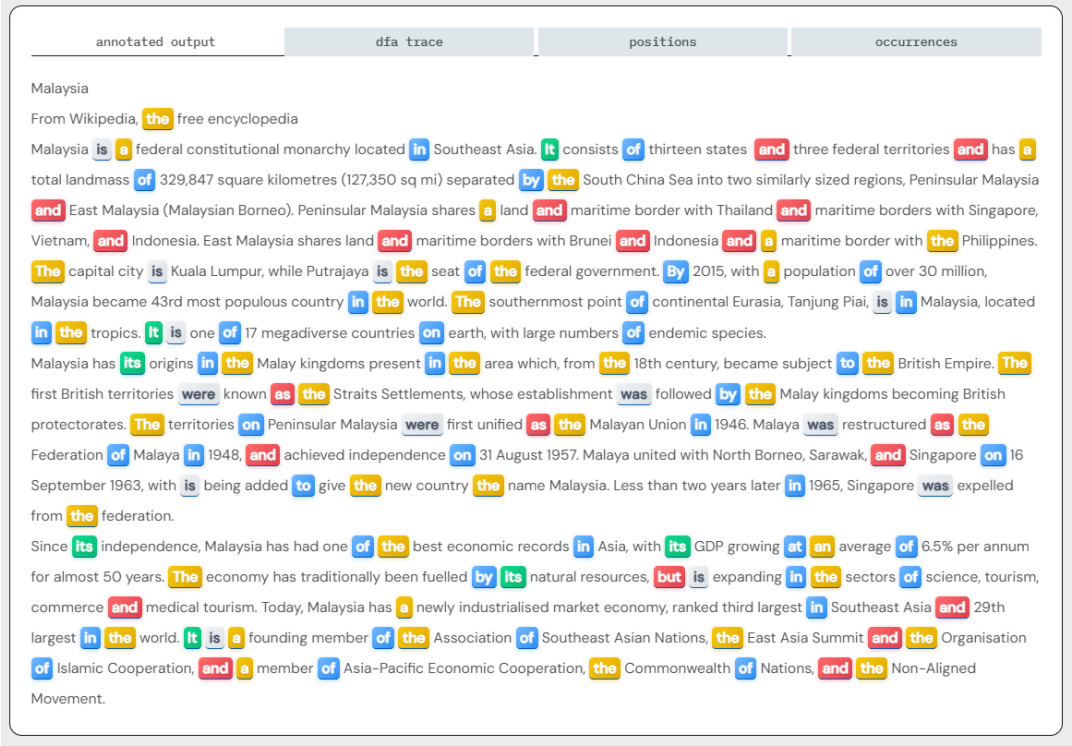

Figure 3 Result of Annotated Tab

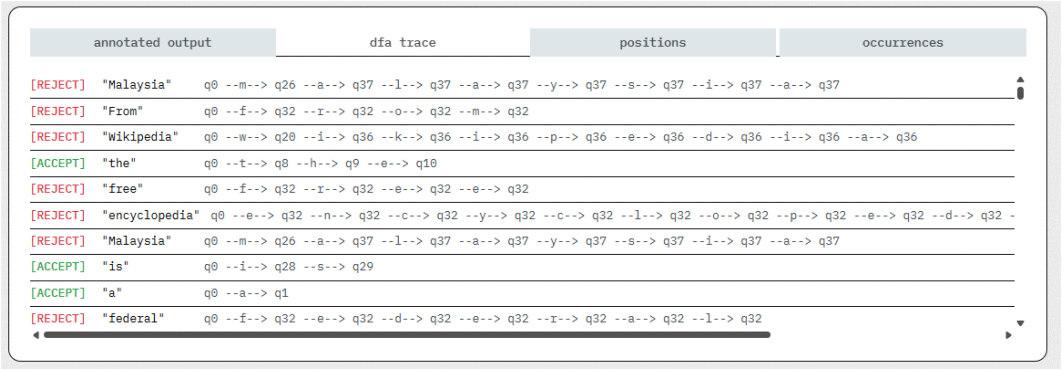

Figure 4 Result of DFA Trace Tab

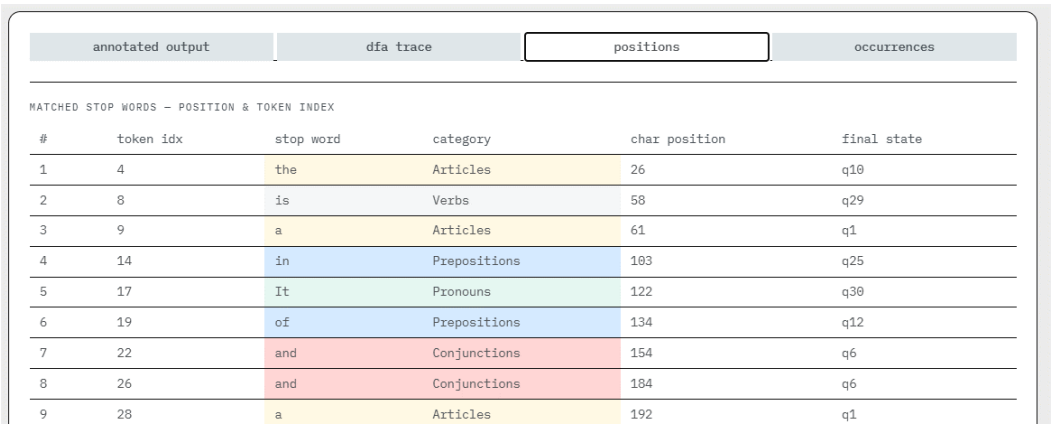

Figure 5 Result of Position Tab

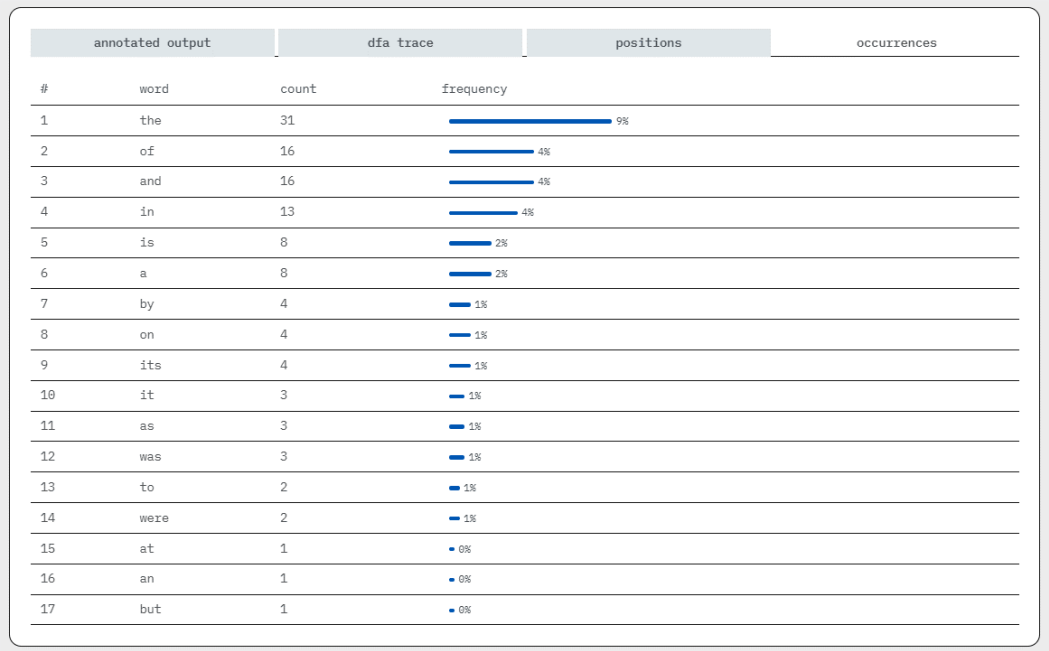

Figure 6 Result of Occurrences Tab

4.4. Appendix D: Sample Text Files Used for Testing

All three texts are supported by the demo interface. Any custom text may also be pasted directly into the input field and processed immediately.

a) Sample Text I: Malaysia Wikipedia excerpt

Malaysia

From Wikipedia, the free encyclopedia

Malaysia is a federal constitutional monarchy located in Southeast Asia. It consists of thirteen states and three federal territories and has a total landmass of 329,847 square kilometres (127,350 sq mi) separated by the South China Sea into two similarly sized regions, Peninsular Malaysia and East Malaysia (Malaysian Borneo). Peninsular Malaysia shares a land and maritime border with Thailand and maritime borders with Singapore, Vietnam, and Indonesia. East Malaysia shares land and maritime borders with Brunei and Indonesia and a maritime border with the Philippines. The capital city is Kuala Lumpur, while Putrajaya is the seat of the federal government. By 2015, with a population of over 30 million, Malaysia became 43rd most populous country in the world. The southernmost point of continental Eurasia, Tanjung Piai, is in Malaysia, located in the tropics. It is one of 17 megadiverse countries on earth, with large numbers of endemic species.

Malaysia has its origins in the Malay kingdoms present in the area which, from the 18th century, became subject to the British Empire. The first British territories were known as the Straits Settlements, whose establishment was followed by the Malay kingdoms becoming British protectorates. The territories on Peninsular Malaysia were first unified as the Malayan Union in 1946. Malaya was restructured as the Federation of Malaya in 1948, and achieved independence on 31 August 1957. Malaya united with North Borneo, Sarawak, and Singapore on 16 September 1963, with is being added to give the new country the name Malaysia. Less than two years later in 1965, Singapore was expelled from the federation.

Since its independence, Malaysia has had one of the best economic records in Asia, with its GDP growing at an average of 6.5% per annum for almost 50 years. The economy has traditionally been fuelled by its natural resources, but is expanding in the sectors of science, tourism, commerce and

medical tourism. Today, Malaysia has a newly industrialised market economy, ranked third largest in Southeast Asia and 29th largest in the world. It is a founding member of the Association of Southeast Asian Nations, the East Asia Summit and the Organisation of Islamic Cooperation, and a member of Asia-Pacific Economic Cooperation, the Commonwealth of Nations, and the Non-Aligned Movement.

b) Sample Text II: MLKgraphs2021 international workshop call for papers

The 3rd International Workshop on Machine Learning and Knowledge Graphs (MLKgraphs2021) September 27 - 30, 2021 - Linz, Austria

http://www.dexa.org/mlkgraphs2021

email: dexa@iiwas.org

Papers submission: https://easychair.org/conferences/?conf=mlkgraphs2021

**** IMPORTANT DATES ****

Paper submission: April 23, 2021 (SHARP)

Notification of acceptance: June 1, 2021

Camera-ready copies due: June 30, 2021

*** PUBLICATION ***

All accepted papers will be published by Springer in "Communications in Computer and Information Science".

*** SCOPE ***

Knowledge Graphs are becoming a key technology for large-scale information processing systems containing massive collections of interrelated facts. Specifically, Knowledge Graphs provide the means for development of the newest data methods for data management, data fusion, data merging, and graph optimization and modeling, serving as a source of high quality data and a base for web-scale information integration.

The 3rd International Workshop on Machine Learning and Knowledge Graphs aims to be a meeting point for researchers and practitioners working on the latest advances in the intersection of machine learning technologies and knowledge graphs. Therefore, we welcome submissions of novel research that brings together the two topics of Machine Learning (ML) and Knowledge Graphs (KGs) either applying ML models for semantic data management structures (like KGs or ontologies), or by presenting newly assembled Knowledge Graphs that support the task of Machine Learning for certain application domains. Examples areas are Business Analytics, Customer Relationship Management, Fault Detection, Industry 4.0, or Social Networking.

c) Sample Text III: Sushi King restaurant review

Overall, the experience is fair for the price you pay. What makes it meaningful to me is that more than 10 years ago, there weren't many Japanese food options around, and Sushi King was one of the first places I tried. It became the go-to spot for sushi when choices were limited, and it introduced many of us to simple, affordable Japanese cuisine.

The quality of the food is decent, though not exceptional compared to today's wide variety of Japanese restaurants. Still, their menu has been consistent over the years, and the pricing is reasonable for casual dining. Service is usually quick, and the environment feels approachable for both families and groups of friends.